the state of an object, allowing the outside world to interact with the object. These externally visible facets of an object are called its *attributes*.

For our BookInStock objects, the first thing we may need is the ability to find out the ISBN and price (so we can count each distinct book and perform price calculations). One way of doing that is to write *accessor methods*:

tut_classes/stock_stats/book_in_stock_3.rb

```
class BookInStock
  def initialize(isbn, price)
    @isbn = isbn
    @price = Float(price)
  end

  def isbn
    @isbn
  end

  def price
    @price
  end
  # ..
end

book = BookInStock.new("isbn1", 12.34)
puts "ISBN   = #{book.isbn}"
puts "Price  = #{book.price}"
```

produces:

```
ISBN   = isbn1
Price  = 12.34
```

Here we've defined two accessor methods to return the values of the two instance variables. The method isbn, for example, returns the value of the instance variable @isbn, because the last (and only) thing executed in the method is the expression that simply evaluates the @isbn variable. Later, in Method Bodies, on page 86 we'll look at a shorter syntax for declaring one-line methods.

As far as other objects are concerned, there's no difference between calling these attribute accessor methods and any other method. This is great because it means that the internal implementation of the object can change without the other objects needing to be aware of the change.

Because writing accessor methods is such a common idiom, Ruby provides convenient shortcuts. The method attr_reader creates these attribute reader methods for you:

tut_classes/stock_stats/book_in_stock_4.rb

```
class BookInStock
  attr_reader :isbn, :price

  def initialize(isbn, price)
    @isbn = isbn
    @price = Float(price)
  end

  # ..
end

book = BookInStock.new("isbn1", 12.34)
```